

Performance

LG Architecture Training Program

Paulo Merson

Performance



- “Performance” is an umbrella term for various qualities, such as:
 - **Throughput**: number of requests or transactions executed per unit of time
 - **Latency**: time between the arrival of an input/stimulus and the generation of the system response/output
 - Response time, roundtrip time, transaction span are all related terms
 - **Jitter**: variation allowed or observed in the latency
 - **Number of missed events**: events that were not processed in time because the system was busy
 - Aka deadline miss ratio

Performance tactics

- Reduce computational overhead
- Increase resources
- Manage work requests (sampling rate or event arrival)
- Introduce concurrency
- Maintain multiple copies of computations
- Maintain multiple copies of data

*We'll discuss a few
selected tactics in class.
See [Bass21] for lists
of tactics per QA*



Reduce computational overhead

- Aka Reduce overhead [Bass13]
- Strategies:
 - Avoid remote calls by collocating components on the same machine, VM or container
 - Do not use intermediaries, such as proxies, gateways, authentication servers, interceptors
 - Clean up resources periodically such as hash tables and memory maps that may become inefficient
- Tradeoffs:
 - Modifiability and reuse
 - Deployability

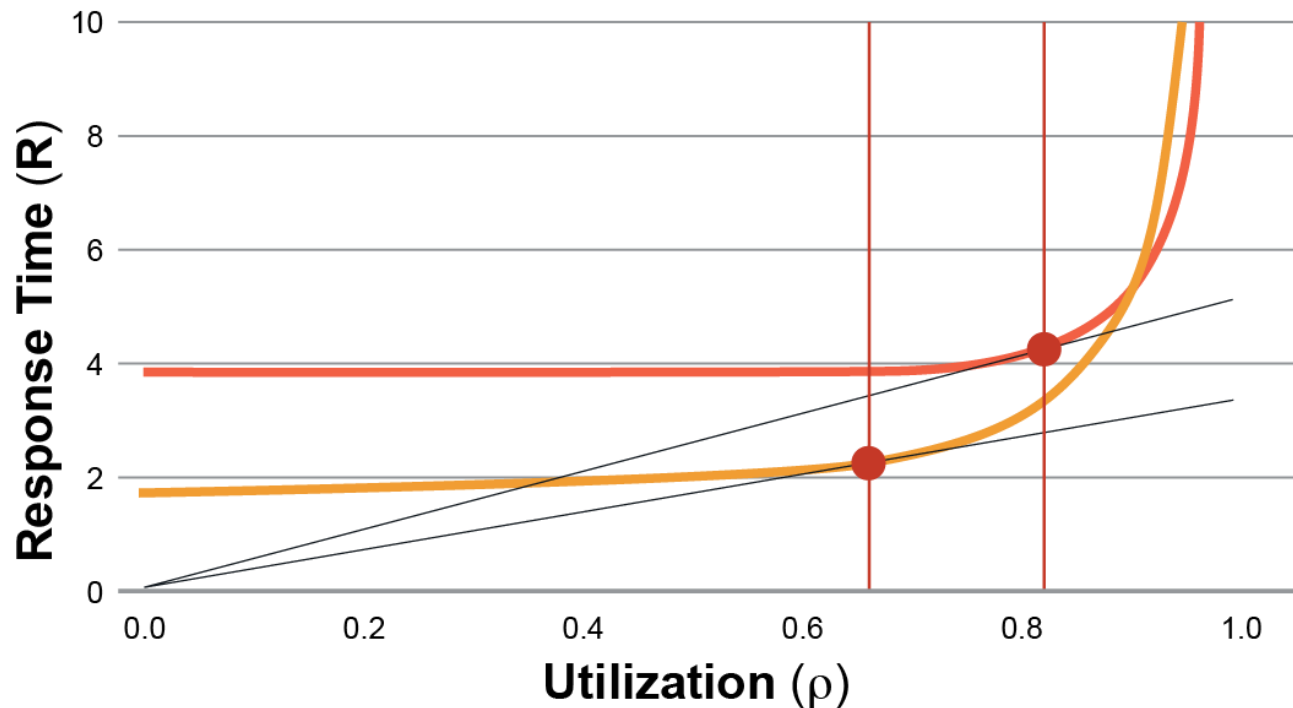
A system reboot is a possible way to "clean up"


Increase resources

- Alternatives:
 - Faster processor/node
 - Faster disk
 - More processors
 - More memory
 - Larger bandwidth
- Tradeoff: cost

Increase resources on server machines

Increase resources is not a solution if utilization is beyond a critical point (“the knee”)



The knee is the utilization value at which response time divided by utilization is at its minimum

Cary Millsap



Manage work requests

- Reduce the number of requests coming into the system
- **Manage sampling rate**
 - Adjust sampling frequency to the response times the computing element offers
 - Ex.: sensor data sampling rate; FPS in video capture or video processing
 - Goal is to balance predictable latency and good enough quality/fidelity
- **Manage event arrival**
 - **API quota**: limit the requests per client per day, week or month to a quota
 - **Rate limiting**: limit the number of requests per client per seconds or per minute
 - **Request throttling**: confine requests to configured limits to avoid overwhelming the service

The token bucket algorithm is often used for request throttling



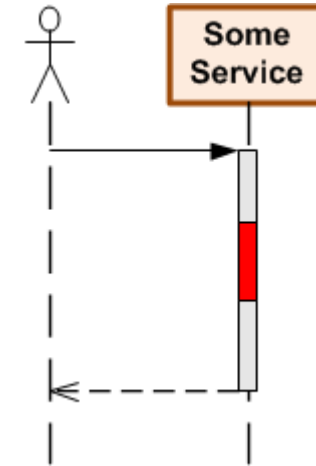
Introduce concurrency

- Reduces response time or latency
- Strategies:
 - Spawn threads
 - Pipeline (pipe and filter)
 - Asynchronous calls
- Tradeoff
 - Modifiability (increased design complexity)

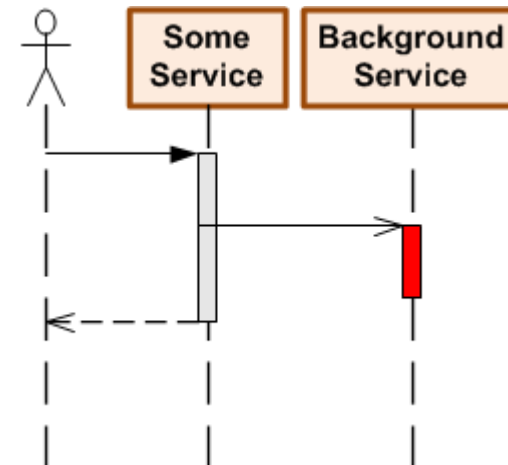
can benefit from multi-core processors

But multi-core is not a requirement

We'll cover sequence diagrams in week 3



WITHOUT concurrency



WITH concurrency

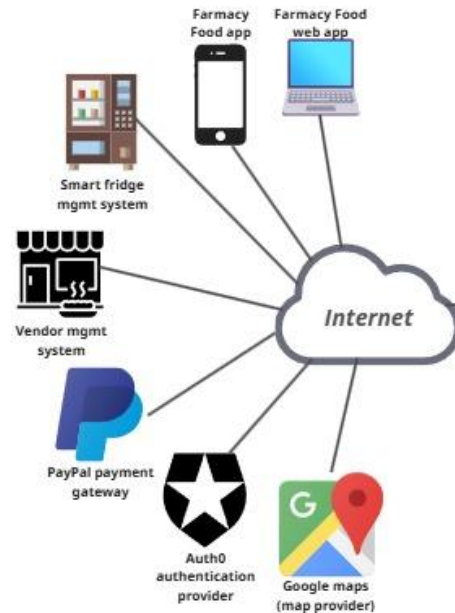
Maintain multiple copies of computations

- Increases throughput but does not affect response time
- Strategy:
 - Component replication + load balancing
- Also improves availability
- Tradeoffs:
 - Cost
 - Deployability

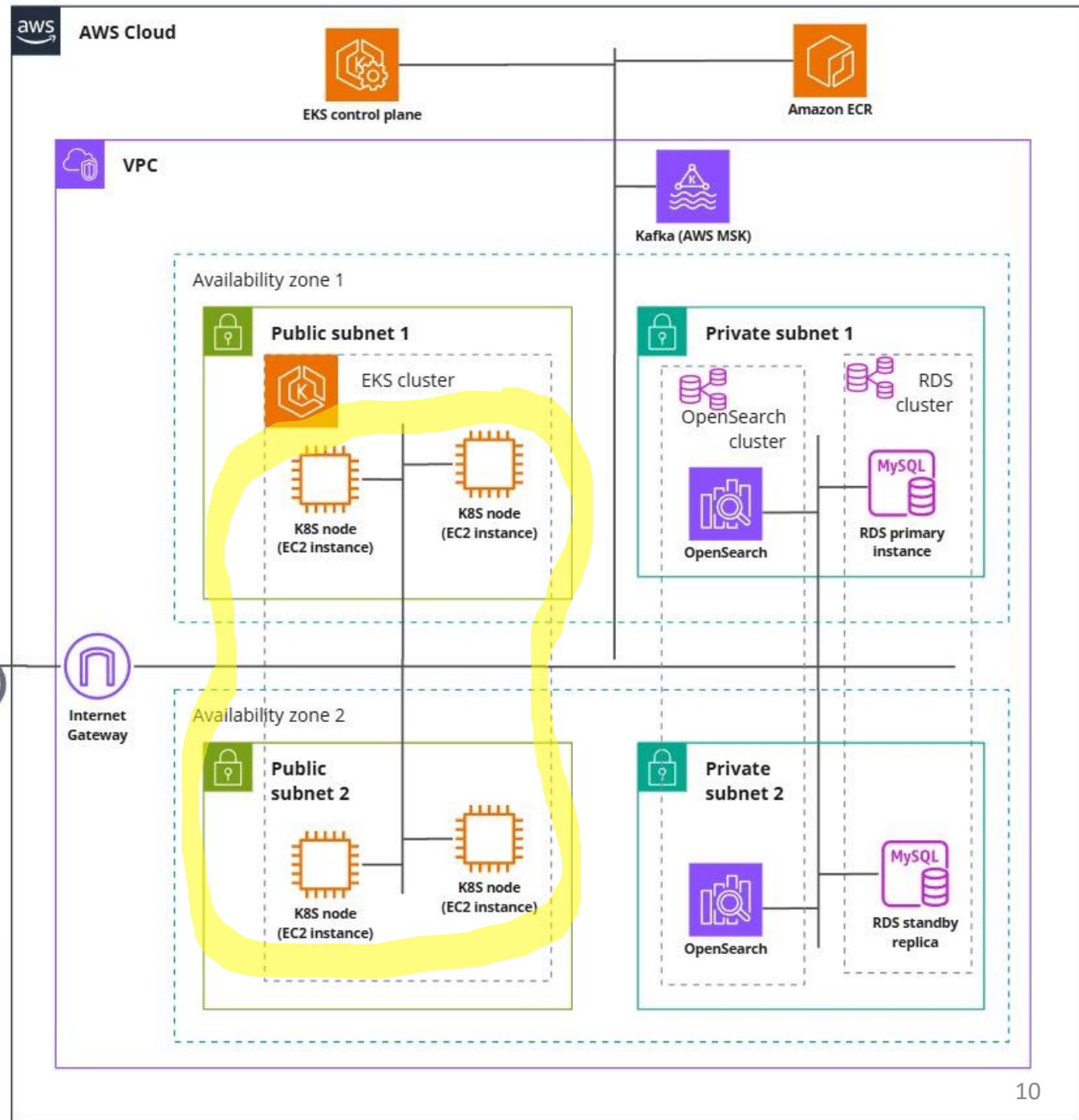
This tactic is problematic for stateful components

See passive redundancy discussed later

Example architecture with component replication



Symbols within the AWS Cloud use standard AWS notation. The lines in the diagram simply represent the network communication channel. Interaction among application components is shown in other diagrams.

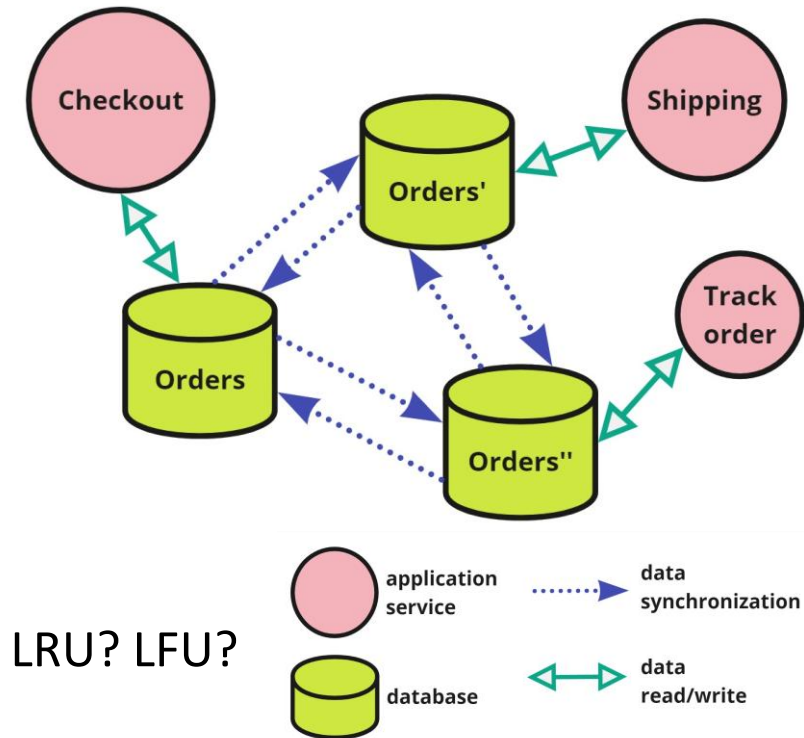


Maintain multiple copies of data

- Strategies
 - Cache in memory
 - Service data replication [Erl09]
- Tradeoffs and considerations
 - Complex implementation
 - Data synchronization mechanism is required
 - Two-way synchronization may be required
 - Risk of operating on stale data due to eventual consistency
 - Cost (especially if multiple DB instances are required)

Data replication design questions

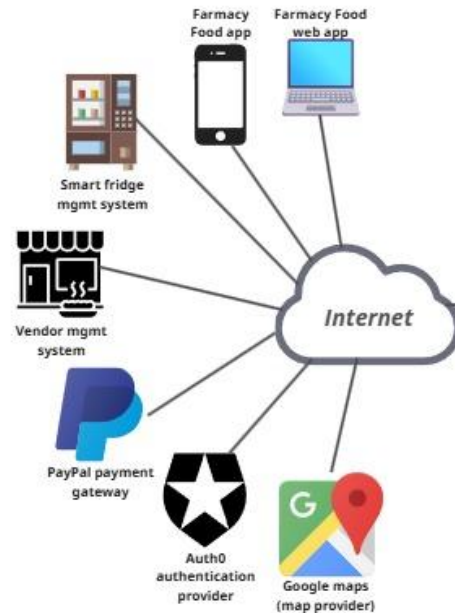
- What's the synchronization mechanism?
 - ETL batch program? How often it runs?
 - Event-based?
- Replica storage design
 - What type of storage?
 - What data format and structure?
- Replicate entire data set?
 - If not, it's like a cache, so what's the cache replacement policy? LRU? LFU?
- What level of replica consistency to enforce?



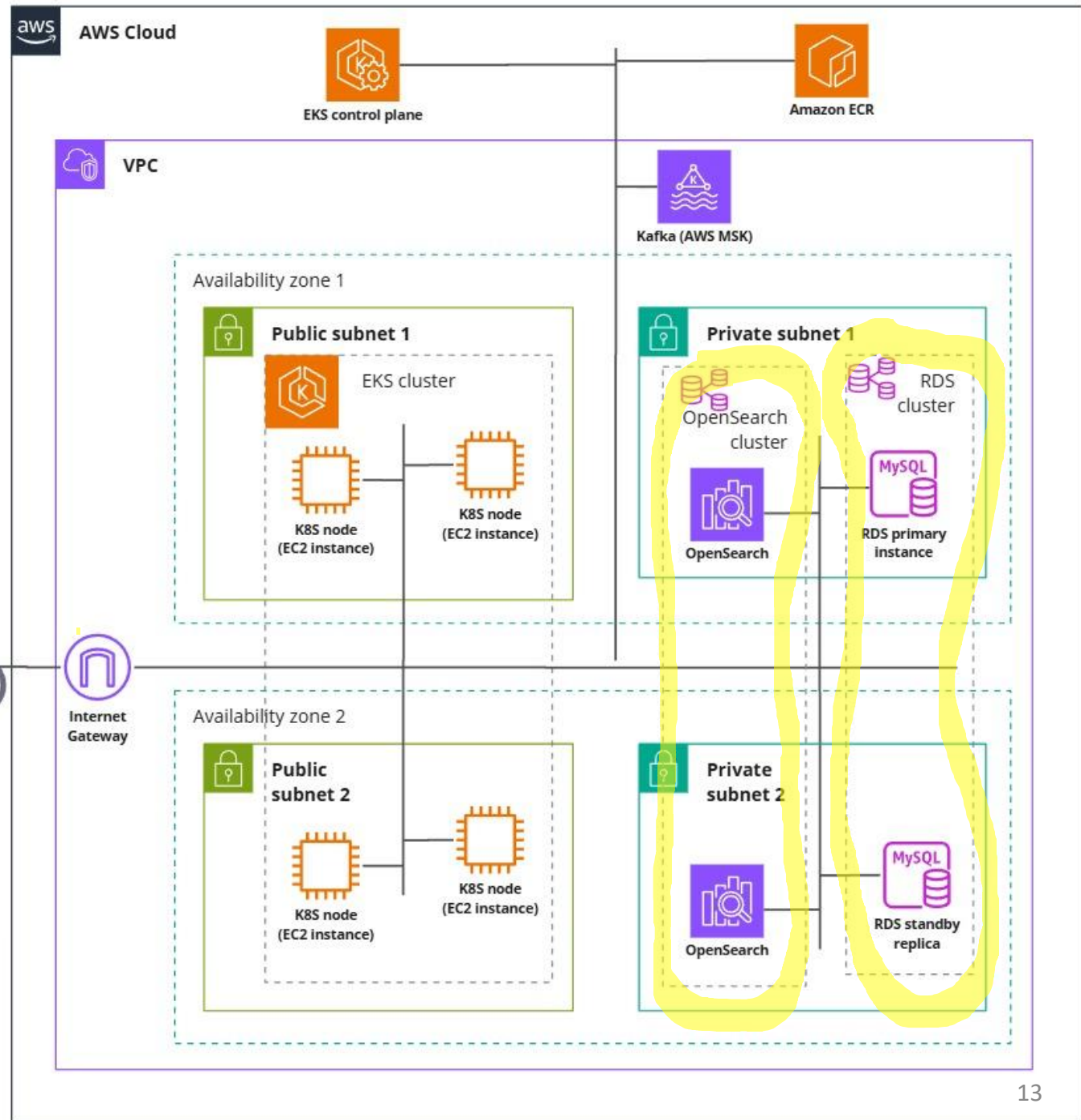
Higher replica consistency == lower availability and performance

We'll revisit
data replication
in week 4

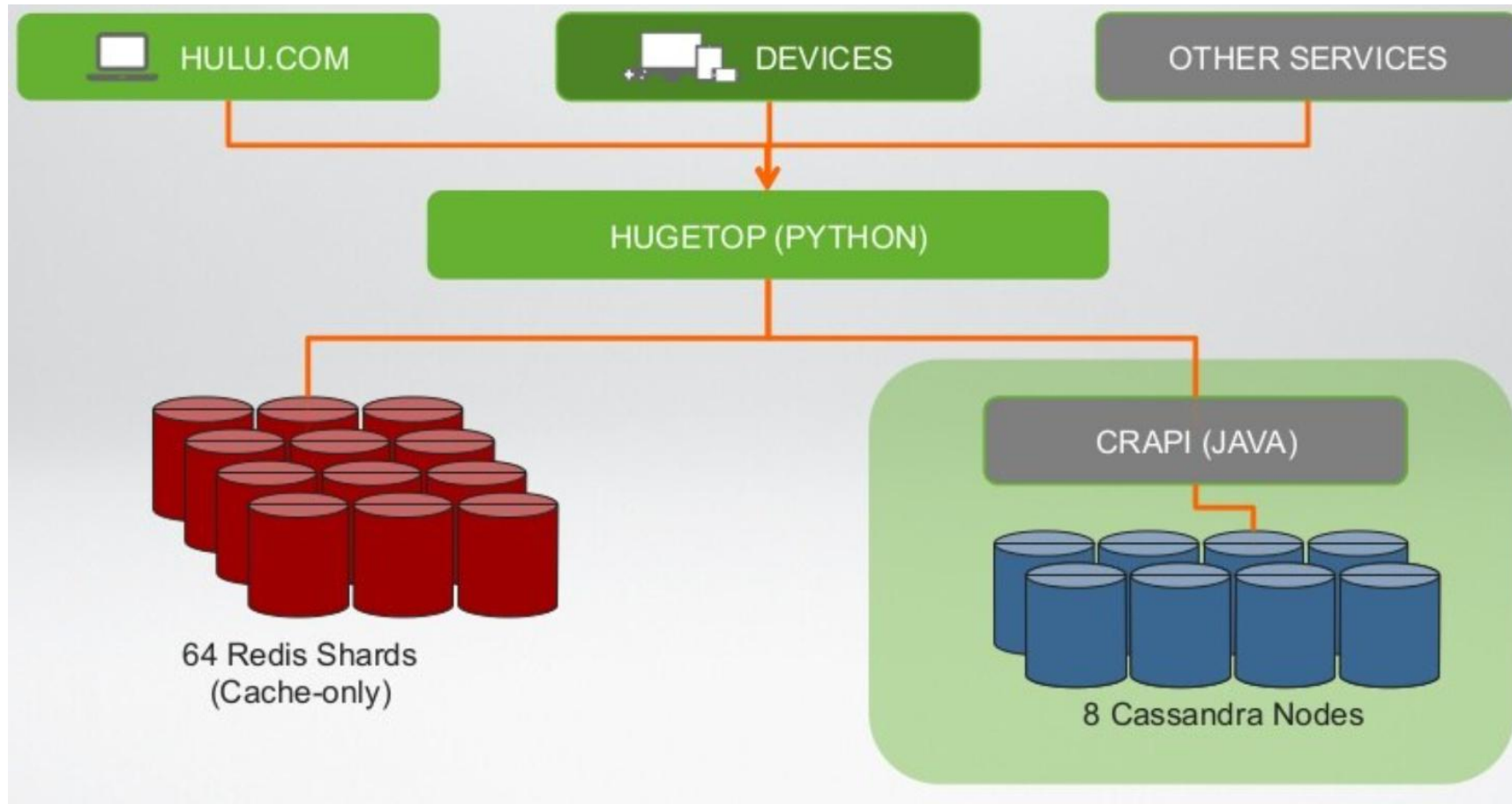
Example architecture with data replication



Symbols within the AWS Cloud use standard AWS notation. The lines in the diagram simply represent the network communication channel. Interaction among application components is shown in other diagrams.



Another example architecture with data replication



Source: slideshare.net/planetcassandra/cassandra-day-sv-2014-scaling-hulus-video-progress-tracking-service-with-apache-cassandra



Questions?

Paulo Merson
pmerson@cmu.edu